Linear regression
Model validation

Vladislav Goncharenko
ML researcher



girafe ai

ITMO, fall 2026

Recap

Lecture 1:
Intro to ML
Naïve Bayes



1. ML and AI overview
2. What is Machine Learning
3. Thesaurus and notation
4. Datasets
5. Exploratory data analysis
6. Maximum Likelihood Estimation
7. Some Machine Learning problems
8. Naïve Bayes classifier
9. Machine learning libraries

Outline

Lecture 2:
Linear Regression



1. Regression model
2. Linear models overview
3. Linear regression
    a. MSE loss
4. Gauss-Markov theorem
    a. BLUE
5. Regularizations
    a. L2, L1, ElasticNet
6. Model validation and evaluation
    a. Parameters and hyperparameters
    b. Dataset splits
    c. Stages organization

Regression model

girafe ai

01

Regression model

Can be written if form

$$Y = f(X) + \varepsilon$$

or equivalently

$$\mathbb{E}(Y|X) = f(X)$$

here we split all the signal measured (Y) into parts containing:

1. Expectation with zero Variance (which we want to predict)
2. Variance with zero Expectation (aka noise)

Regression model assumptions



1. The sample is representative of the population at large
2. The independent variables are measured with no error
3. Deviations from the model have an expected value of zero, conditional on covariates: E(ei|Xi)=0
4. The variance of the residuals ei is constant across observations (homoscedasticity)
5. The residuals ei are uncorrelated with one another. Mathematically, the variance–covariance matrix of the errors is diagonal.

See https://en.wikipedia.org/wiki/Regression_analysis#Underlying_assumptions and https://en.wikipedia.org/wiki/Additive_white_Gaussian_noise

Linear models overview

girafe ai

02

Linear functions aka linear maps



In math is a mapping V → W between two vector spaces that preserves the operations of vector addition and scalar multiplication.

So we have two basic properties defining linear function:

- additivity

$$f(\mathbf{u}+\mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$$

- homogeneity

$$f(c\mathbf{u}) = cf(\mathbf{u})$$

Linear models

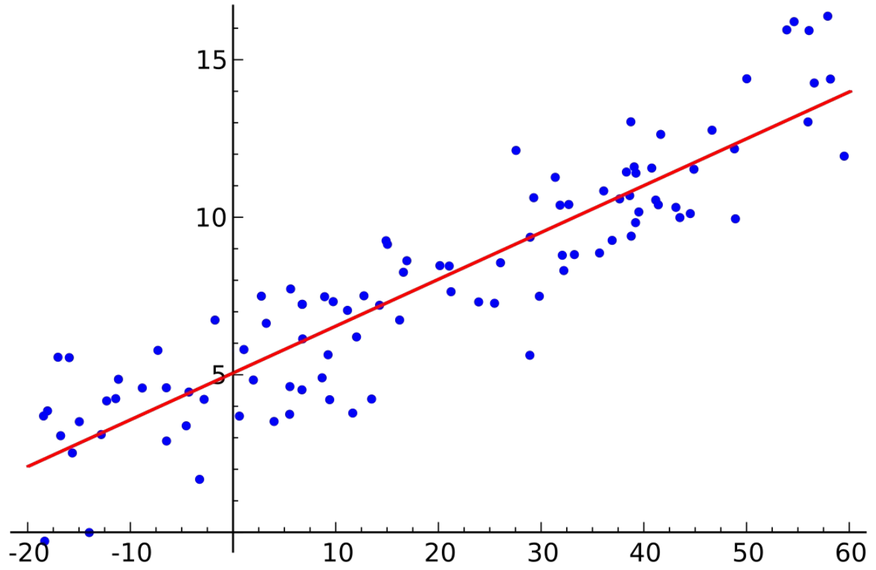
In applied statistics we use linear functions to predict different values

$$Y = X_1 + X_2 + X_3$$

Dependent Variable	Independent Variable
Outcome Variable	Predictor Variable
Response Variable	Explanatory Variable

Linear models

- Regression models



Estimated (or predicted) Y value for observation i

Estimate of the regression intercept

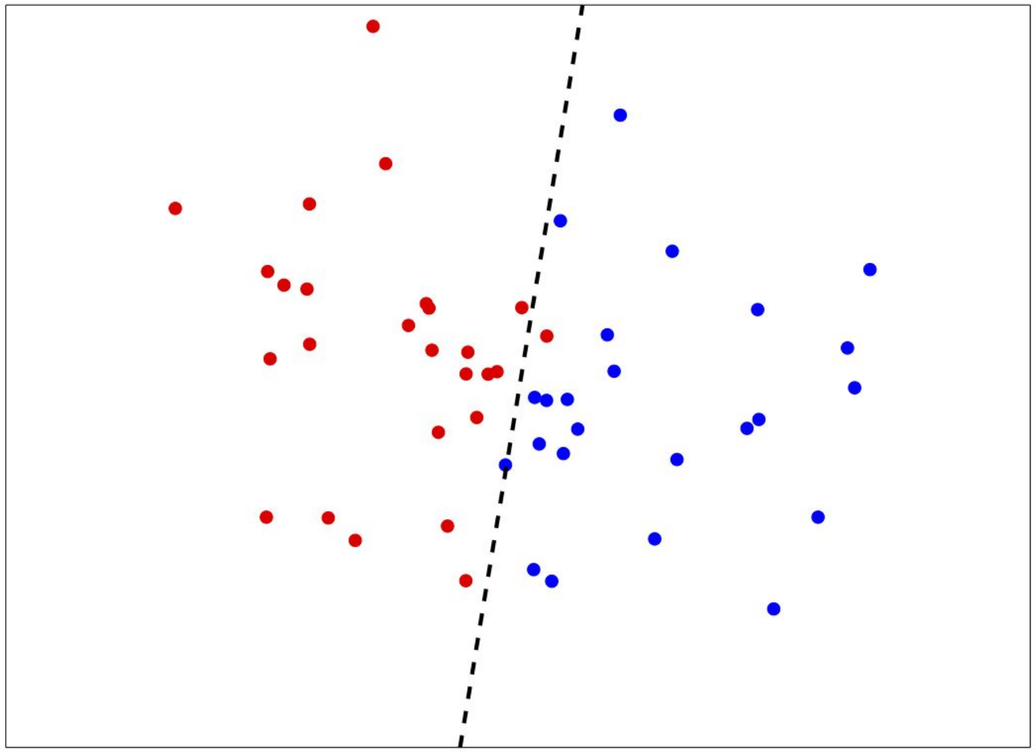
Estimate of the regression slope

Value of X for observation i

$$\hat{Y}_i = b_0 + b_1 X_i$$

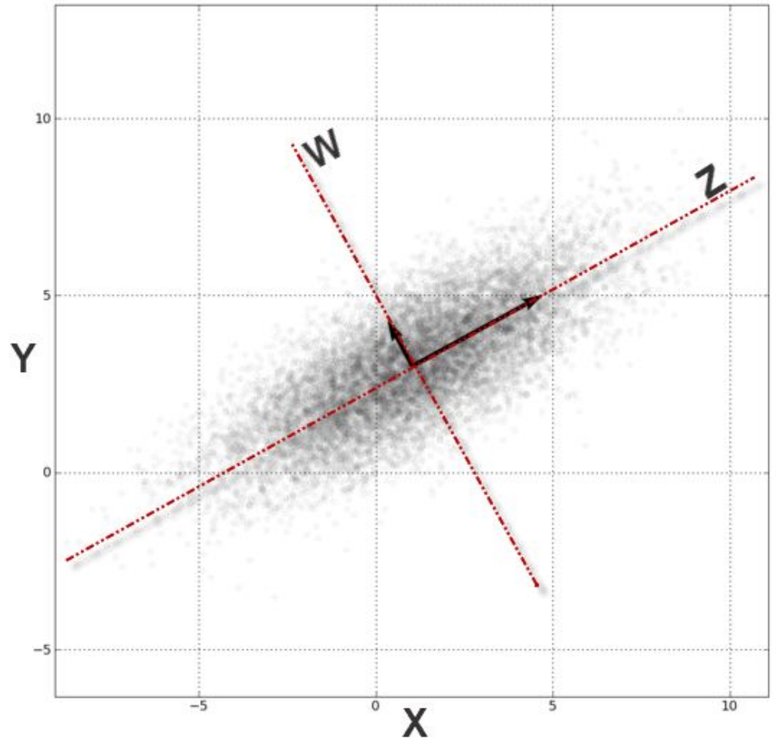
Linear models

- Regression models
- Classification models



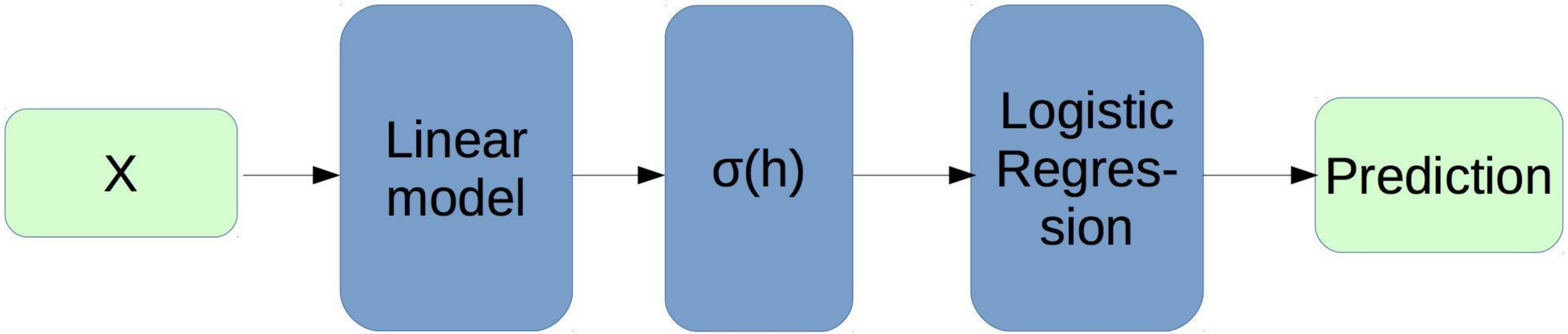
Linear models

- Regression models
- Classification models
- Unsupervised models (e.g. PCA)



Linear models

- Regression models
- Classification models
- Unsupervised models (e.g. PCA)
- Building block of other models (ensembles, NNs, etc.)



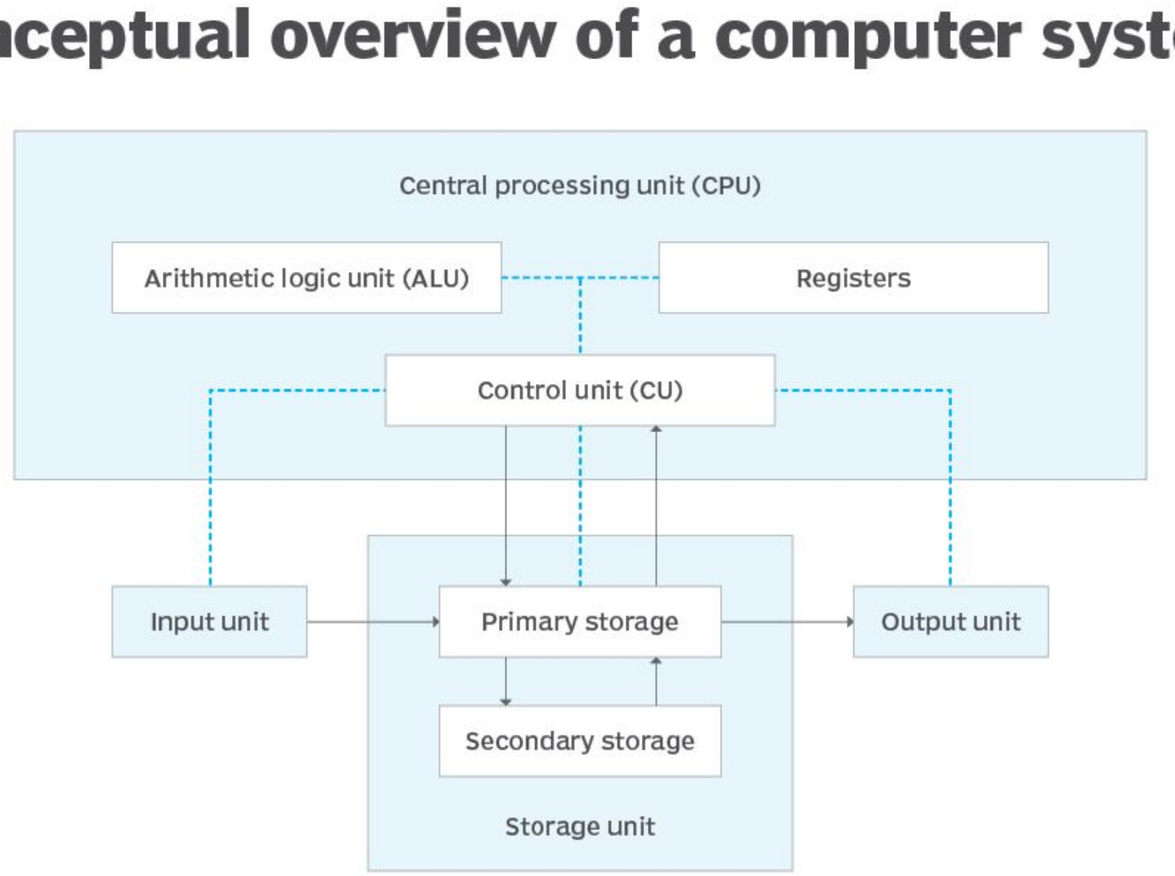
A simple neural network

The only thing calculated efficiently



For computers the only one model we can calculate natively is linear model.

That’s why it is so important to study it in depth!



Linear regression

girafe ai

03

Linear Regression model

When estimator is linear

$$f_w(x) = w_0 + \sum_{i=1}^{p} w_i x_i \equiv x^T w$$

Note: x and w are supposed to include bias term (conventional notation in math)

$$w = (w_0, w_1, \ldots, w_n)^T$$

$$x = (1, x_1, \ldots, w_n)^T$$

Linear Regression problem

Observed samples

$$(x^i, y^i), i = 1, \ldots, n$$

$$x^i \in R^p, y^i \in R$$

Matrix form of dataset

$$X = [x^1, \ldots, x^n]^T, X \in R^{n \times p}$$

$$Y = [y^1, \ldots, y^n]^T, Y \in R^n$$

Linear Regression

$$f_w(X) = Xw = \hat{Y} \approx Y$$

How to find parameters vector?



Apply MLE estimation!

Great visual explanation

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Linear Regression problem

How to choose weights?
With MLE!

$$\text{Empirical risk} = \sum_{\text{by objects}} \text{Loss on object} \rightarrow \min_{\text{model params}}$$

$$Q(X) = \sum_{i=1}^{n} L(y^i, f_w(x^i)) \rightarrow \min_w$$

Loss functions

$$\text{MSE:} \quad L(y_t, y_p) = (y_t - y_p)^2$$

$$\text{MAE:} \quad L(y_t, y_p) = |y_t - y_p|$$

MSE minimization equivalents Maximum Likelihood Estimation in certain conditions (e.g. Gaussian noise)

Linear Regression analytical solution



For MSE closed form solution exists

$$Q_{\text{MSE}}(X) = \sum_{i=1}^{n}(y^i - f_w(x^i))^2 = ||Y - Xw||^2 = (Y - Xw)^T(Y - Xw) \to \min_w$$

$$\nabla_w Q(X) = \nabla_w(Y^TY - (Xw)^TY - Y^TXw + (Xw)^TXw) =$$
$$= 0 - Y^TX - Y^TX + 2X^TXw^T = 0$$

$$w^* = (X^TX)^{-1}X^TY$$

Gauss-Markov theorem

girafe ai

04

Gauss-Markov theorem

$$Y = f(X) + \varepsilon$$

$$f_w(x) = w_0 + \sum_{i=1}^{p} w_i x_i$$

$$\mathbb{E}(\varepsilon_i) = 0 \quad \forall i$$

$$\mathrm{Var}(\varepsilon_i) = \sigma^2 < \inf \quad \forall i$$

$$\mathrm{Cov}(\varepsilon_i, \varepsilon_j) = 0 \quad \forall i \neq j$$

Minimizing MSE loss gives

Best Linear Unbiased Estimation (BLUE)

Estimator with minimal Variance from all unbiased linear estimators

Proof

$$w^* = (X^T X)^{-1} X^T Y$$

$$\mathbb{E}(w^*) = w_{\mathrm{true}}$$

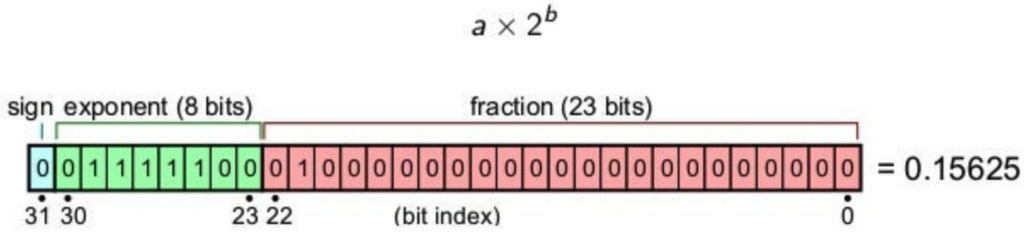
$$Var(w^*) = min$$

Problems with matrix solution

- Computation complexity
  - ~n^3 for n*n matrix
- Unstable inversion result
  - fixed point arithmetics
  - multicollinear features

$$w^* = (X^T X)^{-1} X^T Y$$

So no one uses such way to find solution.



Examples of multicollinear features



- For person
  - weight and height
  - age and years of experience
- For house
  - square and number of rooms
- For weather
  - temperature and date
- For transaction
  - start and end of transaction

Unstable solution

In case of multicollinear features the matrix $X^T X$ is almost singular .

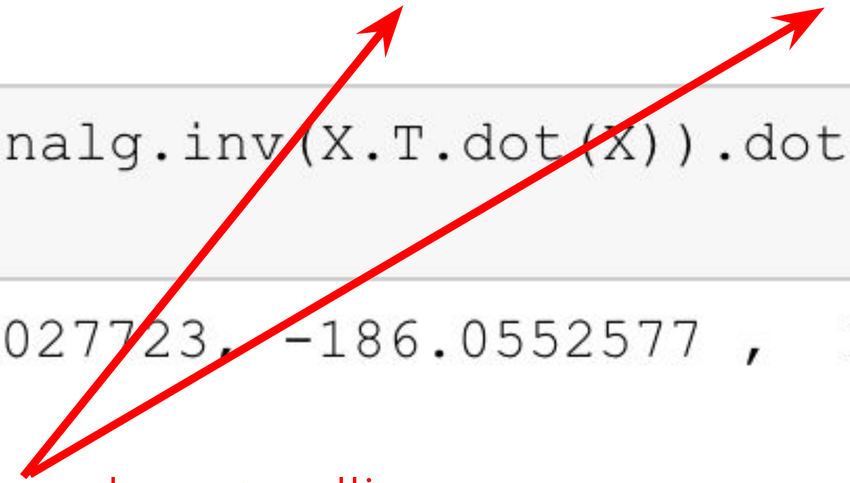
It leads to unstable solution:

```
w_true
```

```
array([ 2.68647887, -0.52184084, -1.12776533])
```

```
w_star = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(Y)
w_star
```

```
array([   2.68027723, -186.0552577 ,  184.41701118])
```



corresponding features are almost collinear

Unstable solution

In case of multicollinear features the matrix $X^T X$ is almost singular .

It leads to unstable solution:

```
w_true
```

```
array([ 2.68647887, -0.52184084, -1.12776533])
```

```
w_star = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(Y)
w_star
```

```
array([   2.68027723, -186.0552577 ,  184.41701118])
```

the coefficients are huge and sum up to almost 0

Regularizations

girafe ai

05

L2 regularization



To make the matrix nonsingular, we can add a diagonal matrix:

$$w = (X^T X + \lambda^2 I)^{-1} X^T Y$$

It is also called Tikhonov regularization or Ridge regression.

Regularization constraints model weights and decrease them.
Resulting solution is **biased** (doesn't converge to true value of w*).

Actually, it's a solution for the following loss function:

$$L_2 = ||Y - Xw||_2^2 + \lambda^2 ||w||_2^2$$

exercise: derive it by yourself

L2 regularization properties

$$w = (X^T X + \lambda^2 I)^{-1} X^T Y$$

$$L_2 = ||Y - Xw||_2^2 + \lambda^2 ||w||_2^2$$

Properties:

- prevents overfitting
  - e.g. outliers in data will affect solution less
- penalizes weights, so they decrease
  - higher weights receive more decrease

L1 regularization

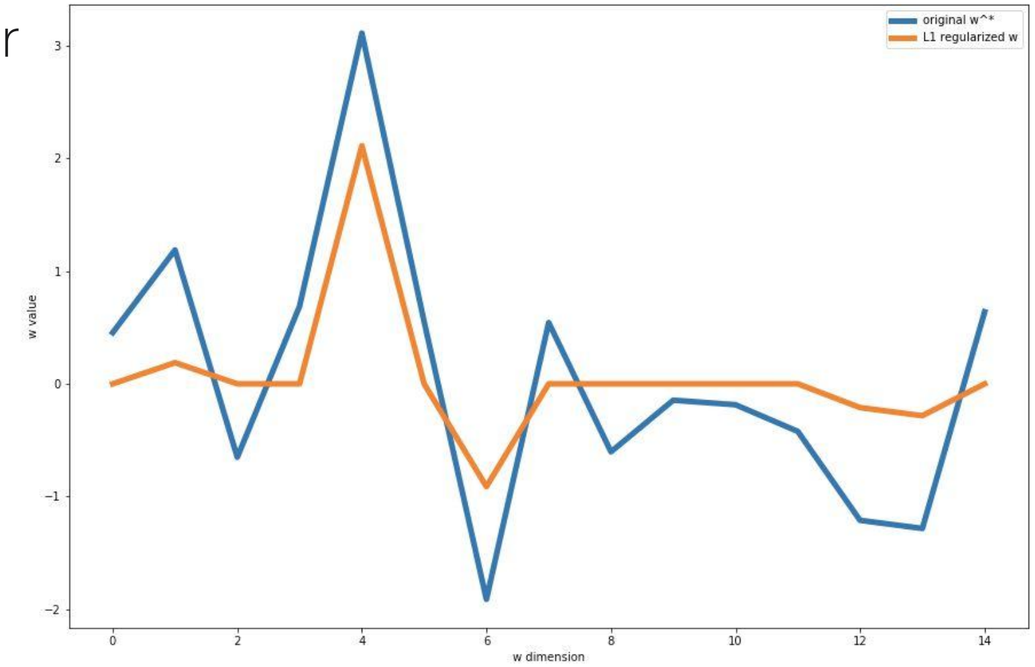
What if we add L1 norm of w to our loss? This technique is called LASSO: least absolute shrinkage and selection operator

$$L_1 = ||Y - Xw||_2^2 + \lambda^2 ||w||_1$$

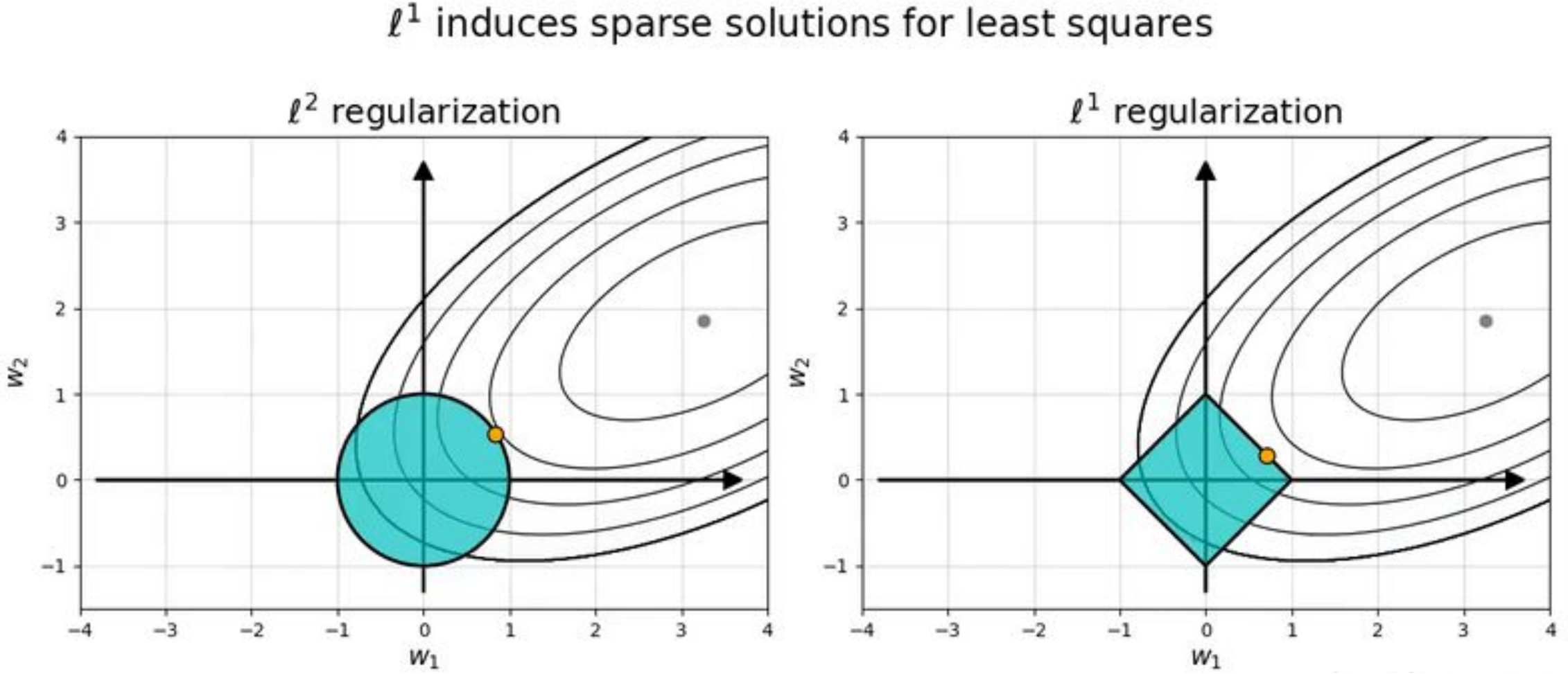
For this case there is no such an elegant solution as for L2 regularization, however solution exists for orthonormal design (see Spokoiny's book p.173)

$$\widehat{\theta}_j = \begin{cases} (\widetilde{\theta}_j - \lambda)_+ & \widetilde{\theta}_j \geq 0, \\ -(|\widetilde{\theta}_j| - \lambda)_+ & \widetilde{\theta}_j < 0 \end{cases}$$

Thus this type of regularization performs implicit feature selection



L1 vs L2 regularization



New optimum point is a balance between original problem and regularization.
New solution is biased!

ElasticNet regularization



Applying both types of regularization also works

$$L_{EN} = ||Y - Xw||_2^2 + \lambda_1^2||w||_1 + \lambda_2^2||w||_2^2$$

It has properties of both types of regularizations at the same time

Regularization of bias term



Let’s return to full definition of linear regression:

$$f_w(x) = w_0 + \sum_{i=1}^{p} w_i x_i \equiv x^T w$$

What should we do with the constant term w_0 in context of regularization? Why?

Bias term is **not penalized** by regularization.

Why this all works?

Regularization could be derived from various considerations.

Neither of these is single source of truth roots of this technique. Each of them is a valid motivation and all of them hold at the same time.

Such type of encounters are covered in classical article **“The Unreasonable Effectiveness of Mathematics in the Natural Sciences”** by Eugene Wigner written in 1960. Text, Wikipedia

“The enormous usefulness of mathematics in natural sciences is something bordering on the mysterious, and there is no rational explanation for it. It is not at all natural that "laws of nature" exist, much less that man is able to discover them. The miracle of the appropriateness of the language of mathematics for the formulation of the laws of physics is a wonderful gift which we neither understand nor deserve”



Loss functions in regression

$$MSE(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N}\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$$

$$MAE(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N}\|\mathbf{y} - \hat{\mathbf{y}}\|_1 = \frac{1}{N}\sum_i |y_i - \hat{y}_i|$$

Different norms

Once more: loss functions:

$$MSE = \frac{1}{n}\|\mathbf{x}^T\mathbf{w} - \mathbf{y}\|_2^2$$

$$MAE = \frac{1}{n}\|\mathbf{x}^T\mathbf{w} - \mathbf{y}\|_1$$

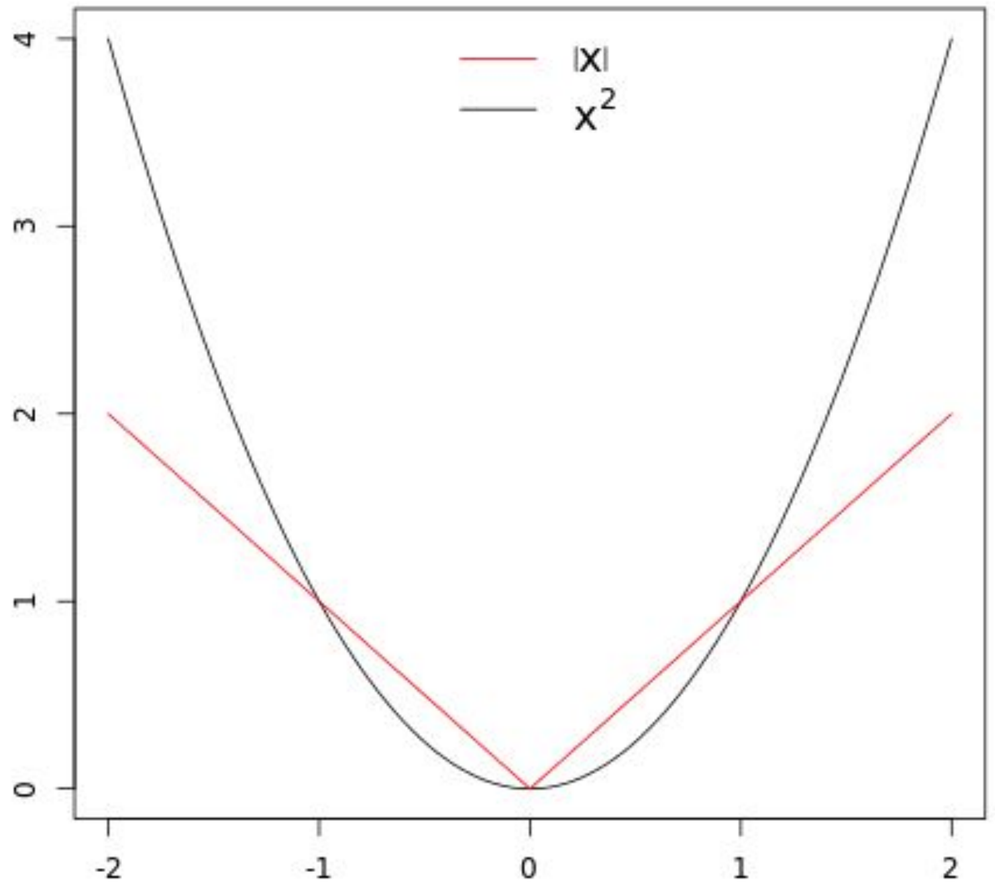
Regularization terms:

- L2 $\|\mathbf{w}\|_2^2$
- L1 $\|\mathbf{w}\|_1$

only works for Gauss-Markov theorem

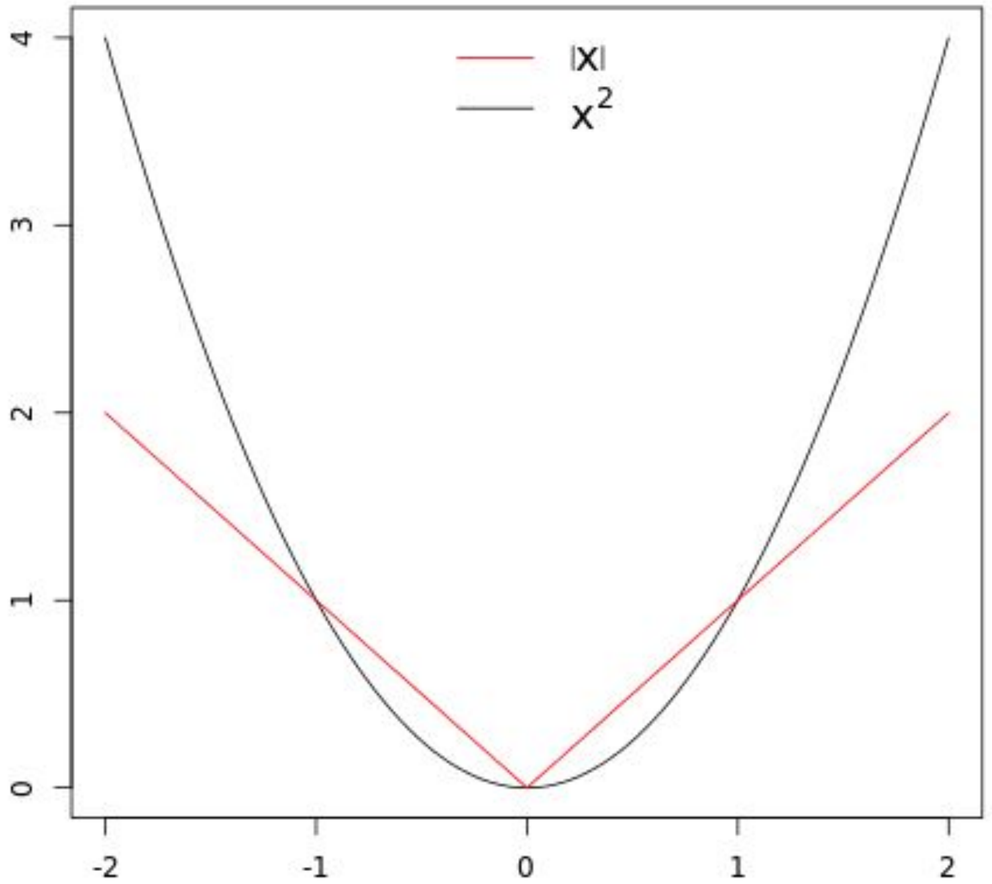
Loss function properties

- MSE (L_2)
  - delivers BLUE according to Gauss-Markov theorem
  - differentiable
  - sensitive to noise
- MAE (L_1)
  - non-differentiable
    - not a problem
  - much more prone to noise



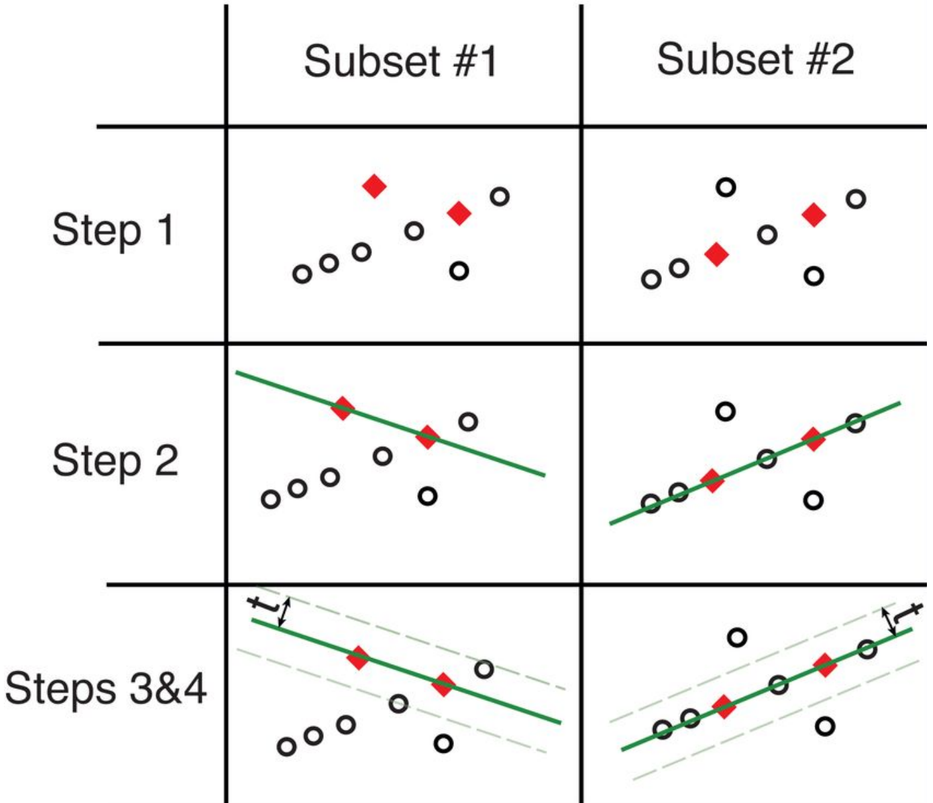
Regularization properties

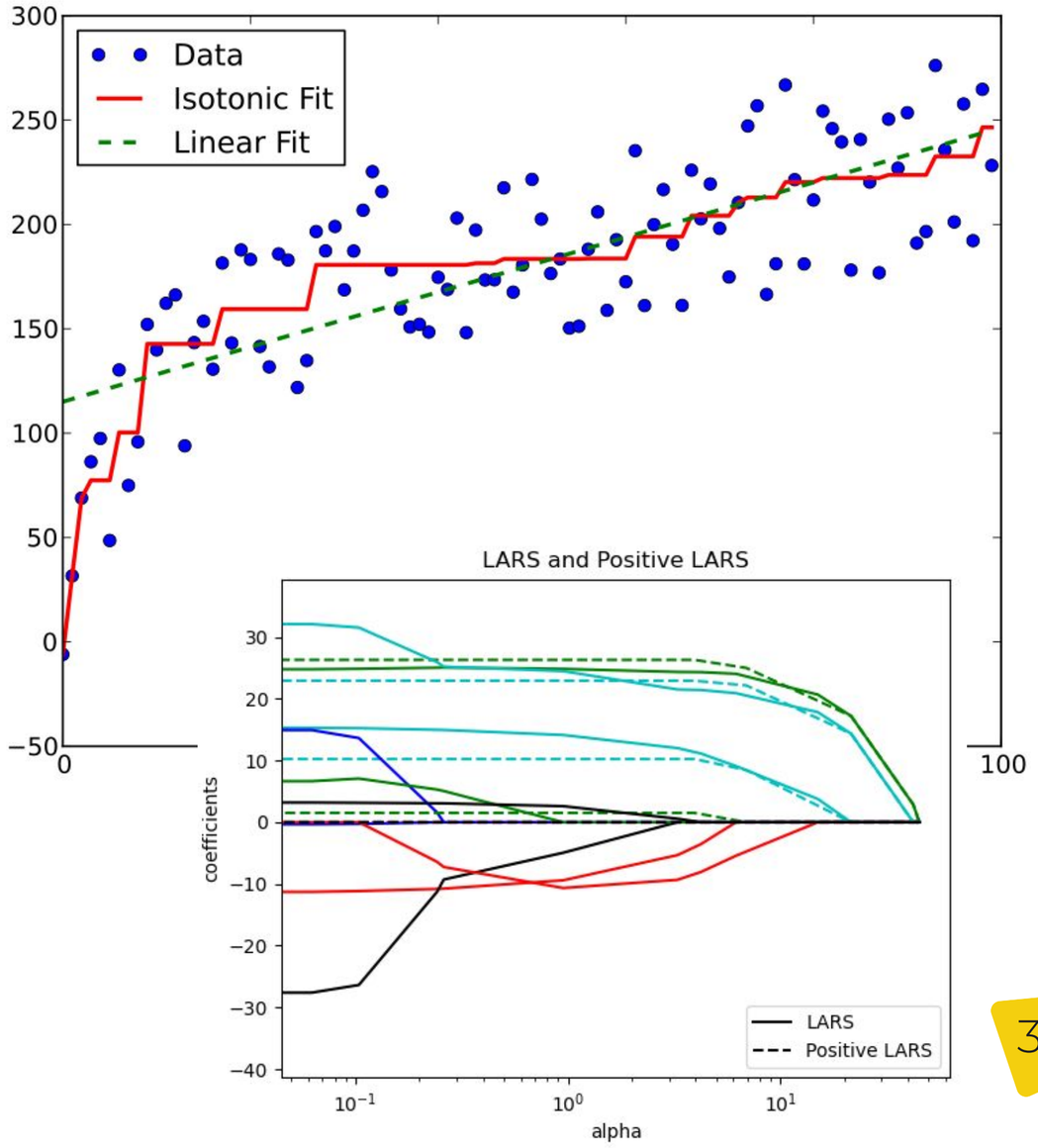
- L_2 regularization
  - constraints weights
  - delivers more stable solution
  - differentiable
- L_1 regularization
  - non-differentiable
  - not a problem
  - selects features



Other types of regression models

- Isotonic regression
- RANSAC
- Least-angle regression
- GLM (video)





A solution for big matrices



Gradient descent

Metrics in regression

girafe ai

06

Metrics in regression



- MSE - Mean Square Error
- MAE - Mean Absolute Error
- RMSE - Root Mean Square Error
- MAPE - Mean Absolute Percentage Error
- SMAPE - Symmetric Mean Absolute Percentage Error
- R2 - “R squared” aka coefficient of determination
- etc… (any combination you like)

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

$$\text{SMAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{2 \cdot |y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

Model validation and evaluation

girafe ai

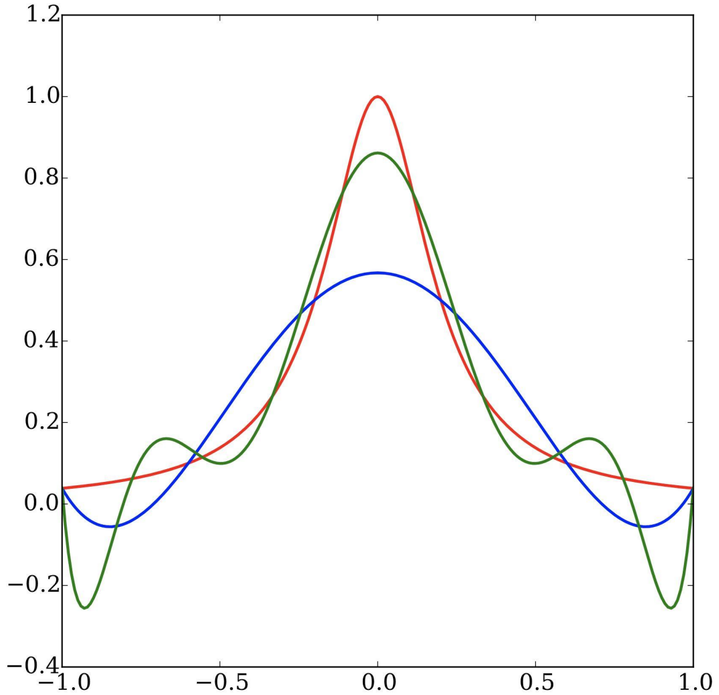
06

Runge's phenomenon

Complex model is not always better!

Runge function interpolation on a uniform grid:

$$f(x) = \frac{1}{1+25x^2}, x \in (-1,1) \quad x_i = \frac{2i}{n} - 1, i = 0, \ldots, n$$



by polynomials of n-th degree

$$P_n(x) = p_n x^n + \cdots + p_1 x + p_0$$

$$P_n(x_i) = f(x_i)$$

is infinitely bad on the whole interval

$$\lim_{n\to\infty} \left(\max_{-1\le x\le 1} |f(x) - P_n(x)| \right) = +\infty$$

Parameters and hyperparameters

The values of **parameters are derived via learning**. In other words parameter values are result of optimization process.

A **hyperparameter** is a constant parameter whose **value is set before the learning** process begins.

Technically any parameter can be made hyperparameter (by fixing it) and vise versa (by estimating hyperparameter from data anyhow).
But for mainstream models there is a stable setup on what is what.

Hyperparameter examples



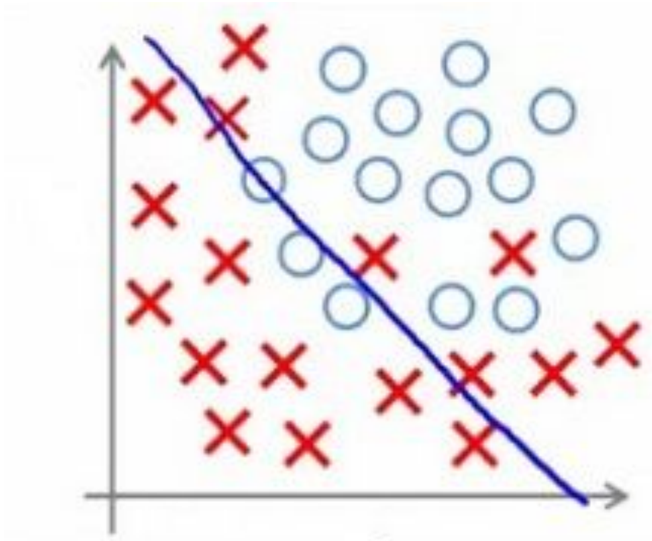
- Naive Bayes classifier - distribution of a feature
  - e.g. Gaussian, Uniform, Multinomial, etc
- Linear regression - regularization type and coefficient
  - e.g. L1, L2 and value of regularization constant
  - learning rate for gradient descent
- Decision trees - max depth of a tree
- Neural networks - number of layers

Parameters vs Hyperparameters



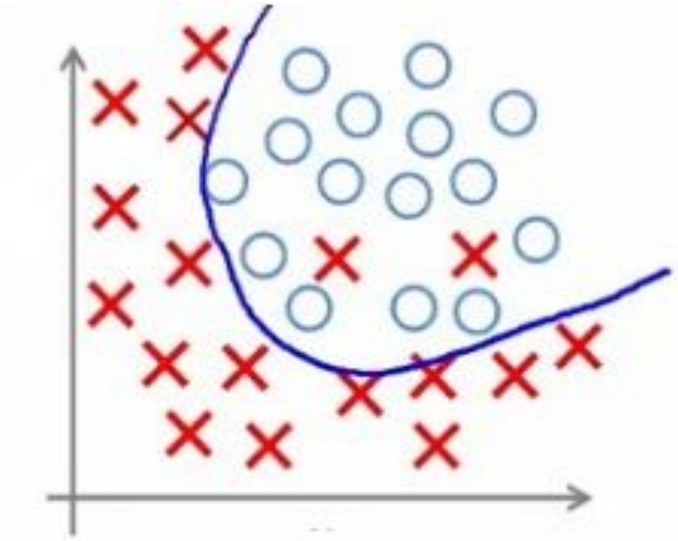
	Defined by	Depend on the training data	Order of optimization methods	Required for	Affect the capacity of the model
Parameters	during the training	yes	first (gradient)	predictions	no
Hyperparameters	before the start of training	no	zero (manual, Bayesian)	training	yes

Underfitting and Overfitting

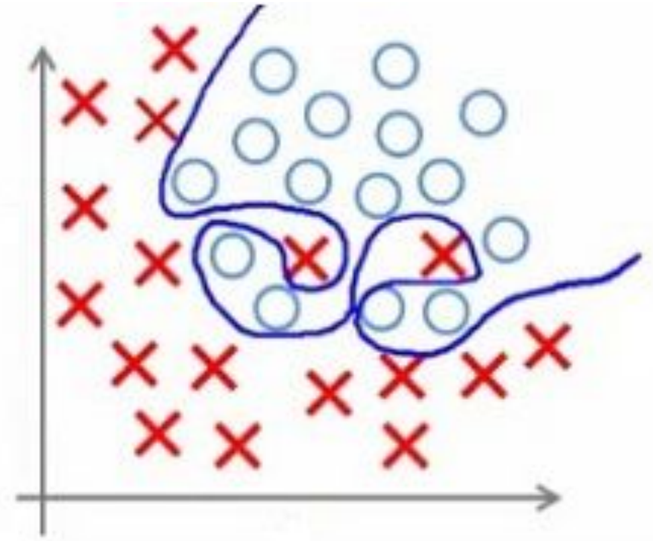


Under-fitting

(too simple to explain the variance)



Appropriate-fitting



Over-fitting

(forcefitting -- too good to be true)

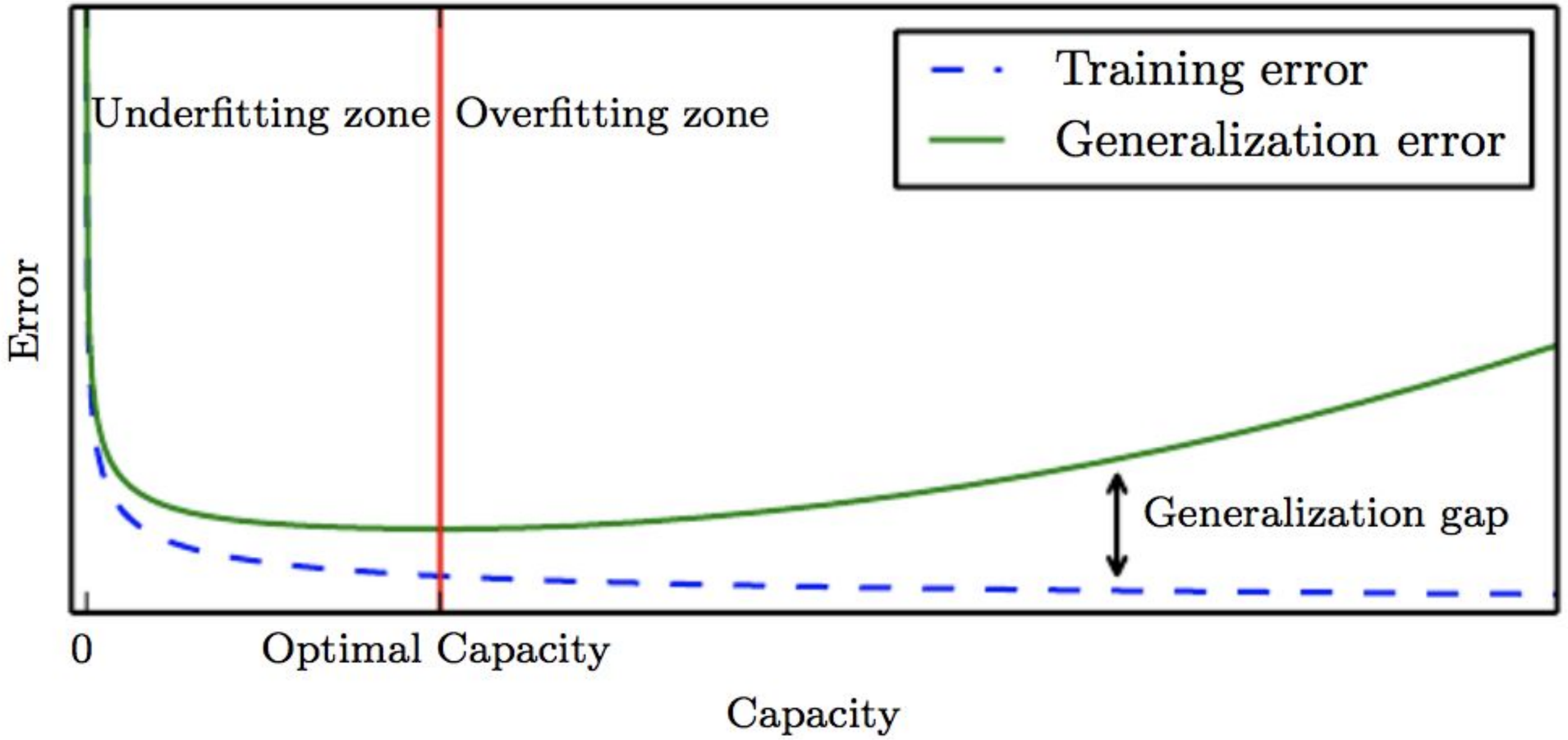
Definitions

Underfitting occurs when a model is too limited or too constrained to learn the true structure of the data, leading to poor performance even on the training set.

Overfitting occurs when a model fits the training data too closely, including noise and accidental peculiarities, leading to overly optimistic training performance but poor generalization to new data.

Model capacity

is any mathematically meaningful measure of the complexity of a hypothesis class (model) that governs its ability to fit data and its tendency to generalize or overfit.



Measuring generalization gap

To make it practical we need to evaluate the quality on unseen data, so we introduce subset of data to train model on and subset for testing.



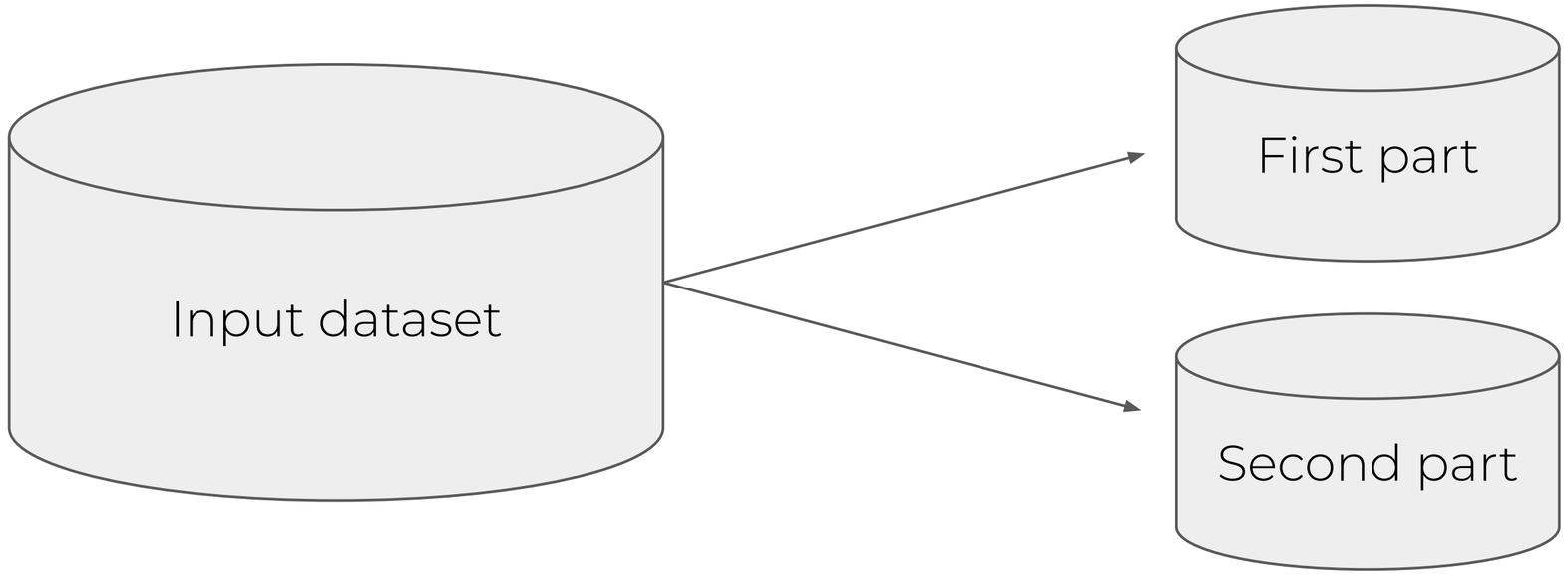
How to split dataset correctly?

Split methods

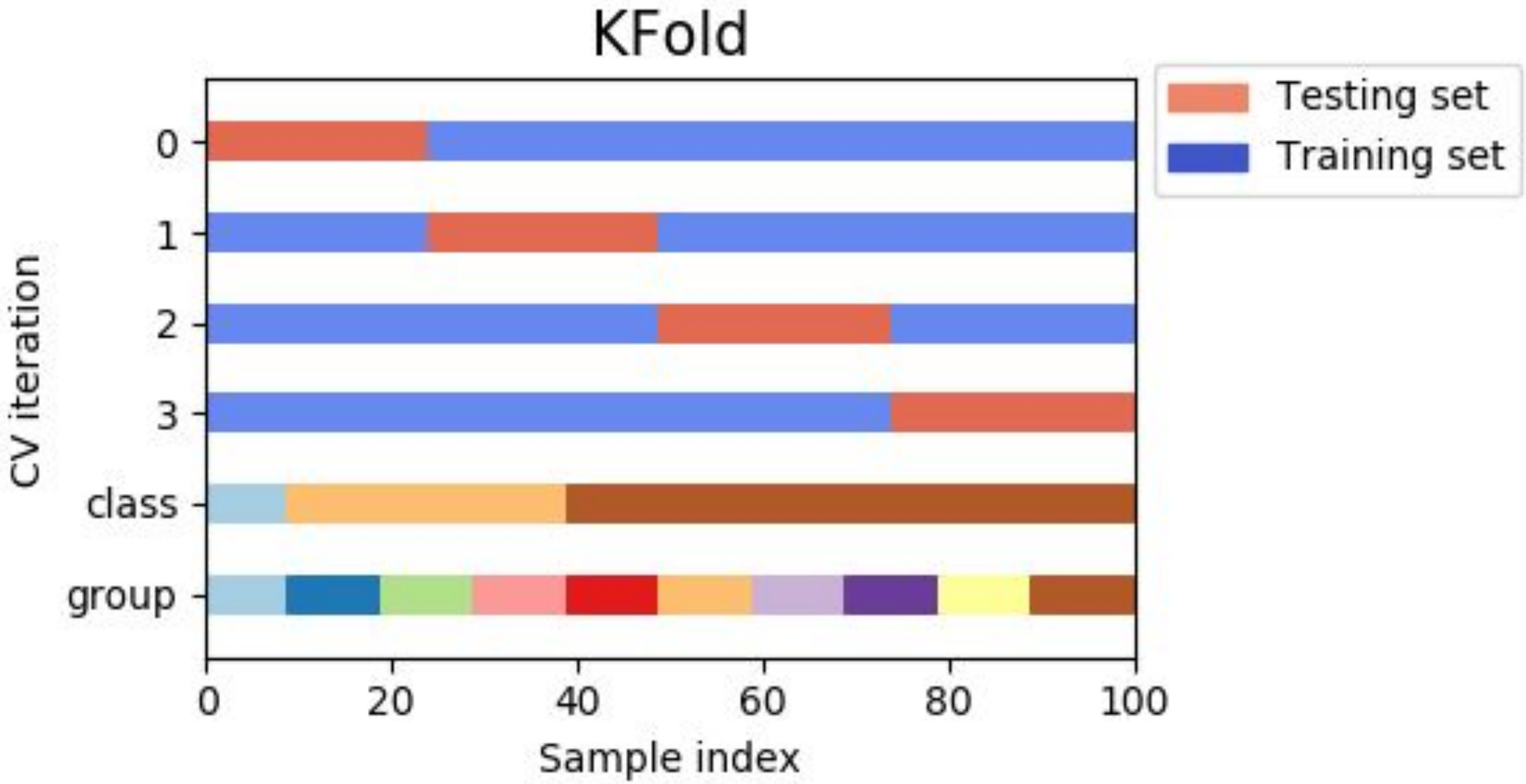
are algorithms to split one set to two sets of predefined size
which is *different from stages organization* approaches described above.

Ideal requirement for a split method is
matching of data distributions in resulting sets.

But in practice we can't check distribution, so we narrow our requirement down to
matching of selected statistics.

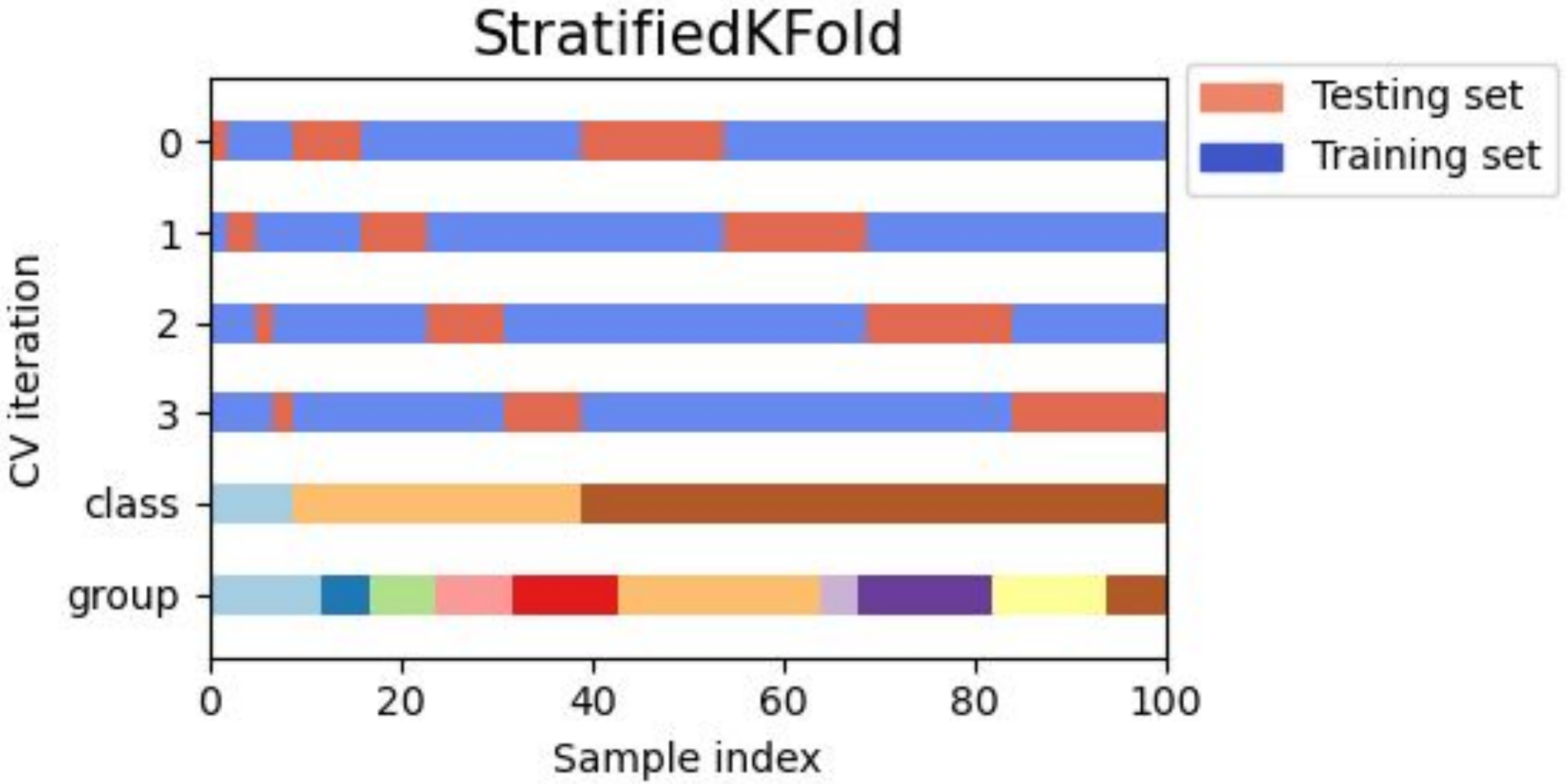


Uniform random split



Baseline method. Word *uniform* is very important!

Stratified split

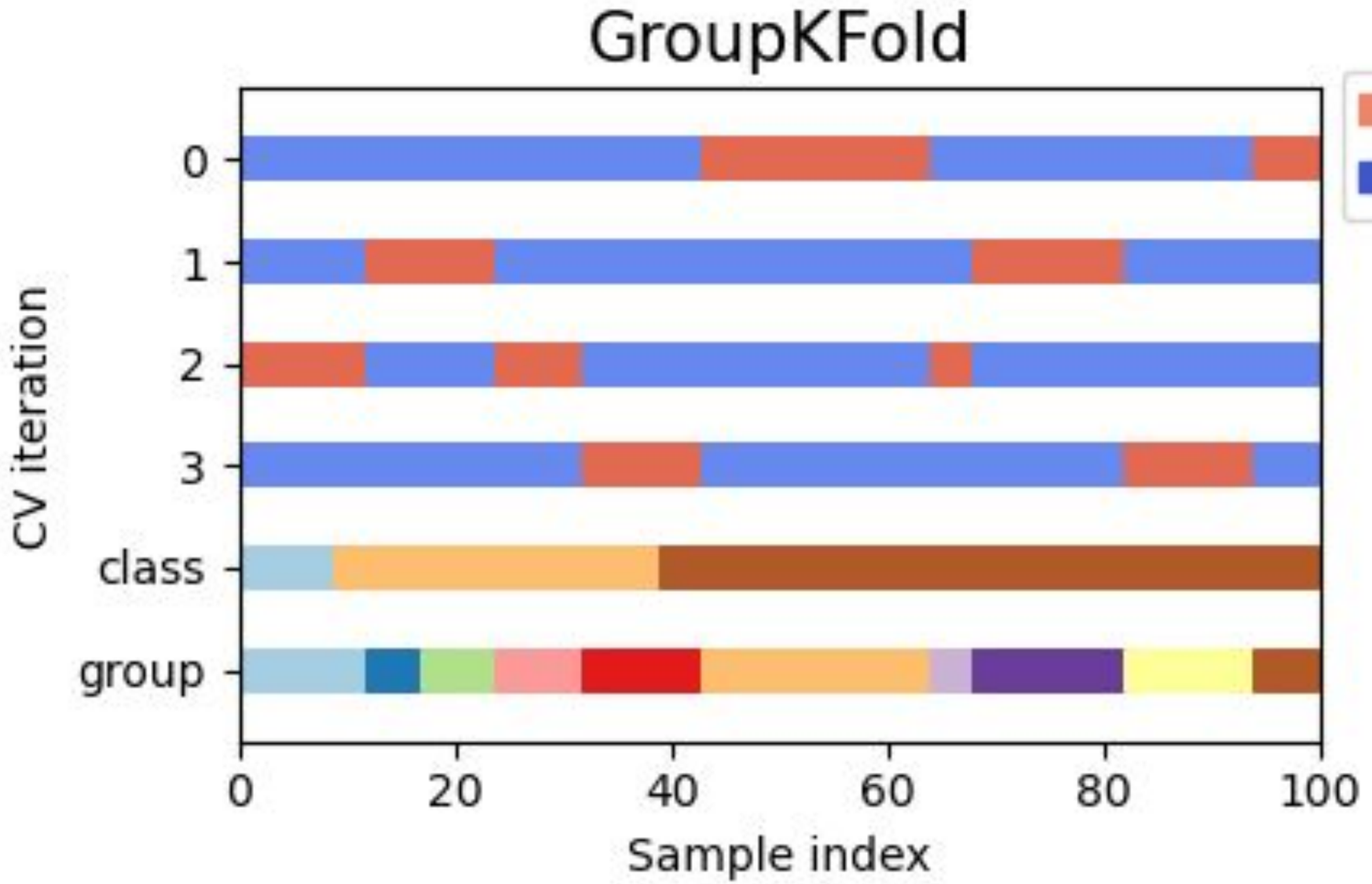


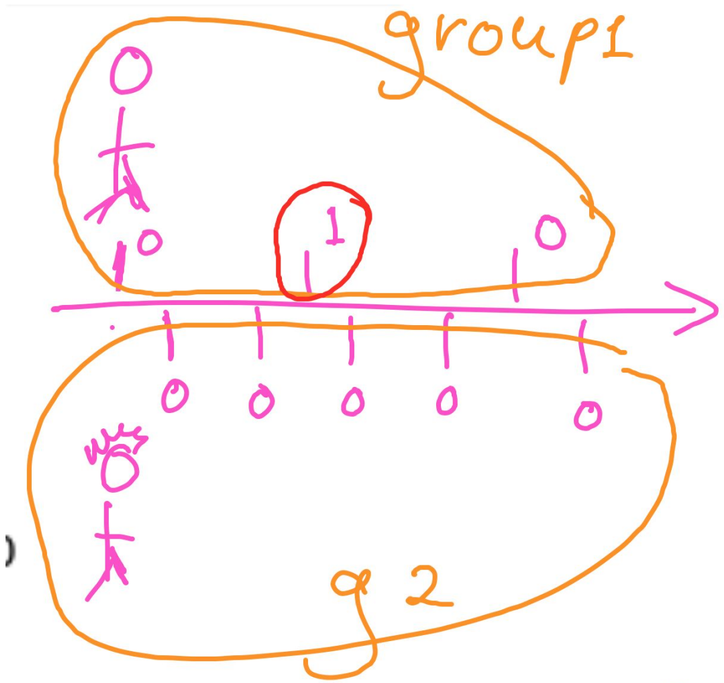
Preserve class ratio for each split. **Default for sklearn** methods

Examples of stratification

- By target
  - positivity of comment on product
  - healthy or ill status
- By feature
  - age category
  - sex

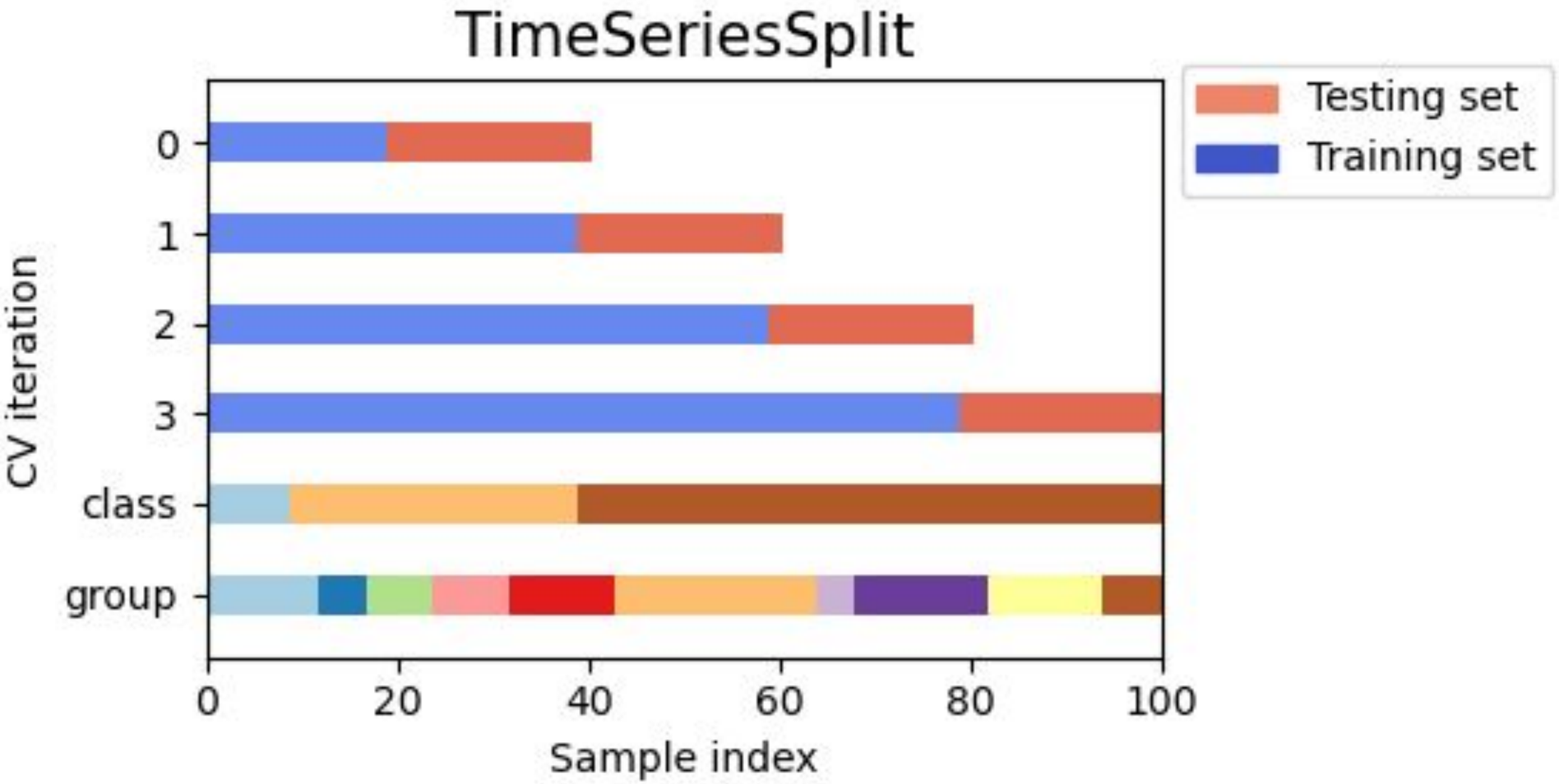
Grouped split





Set whole group either to train or validation.
Used in medicine and ranking (search, recsys)

Timeseries split



Never use train_test_split in this case!!!

Evaluating the quality



Okay, we have splitted train and test correctly



Is this scheme good enough?
Could we somehow overfit?

Evaluation procedure







Hyperparameters | **Parameters** | **Score**

n_layers = 3
n_neurons = 512
learning_rate = 0.1

Weights optimization → 85%

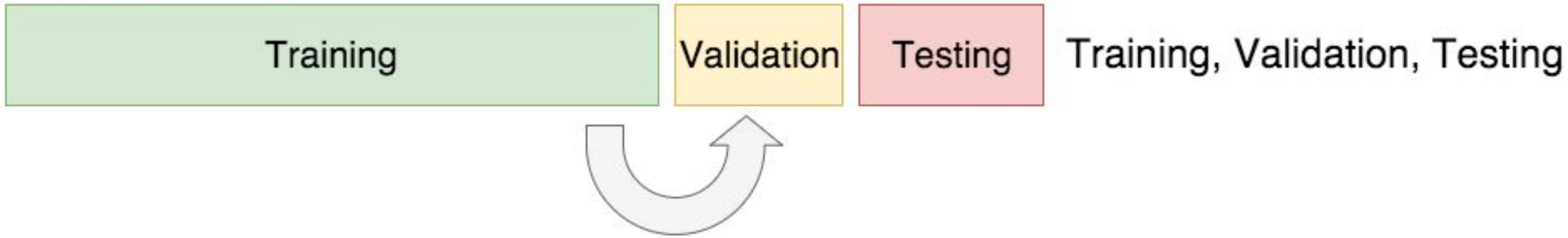
n_layers = 3
n_neurons = 1024
learning_rate = 0.01

Weights optimization → 80%

n_layers = 5
n_neurons = 256
learning_rate = 0.1

Weights optimization → 92%

Necessary dataset splits



Fixed dataset split is the most used in practice nowadays because sizes of dataset are usually big enough.

Model building stages

Split	training	validation	test
Used for	parameters optimization	hyperparameters selection	quality measurement
Overfitting level	high	average	low

Three required steps



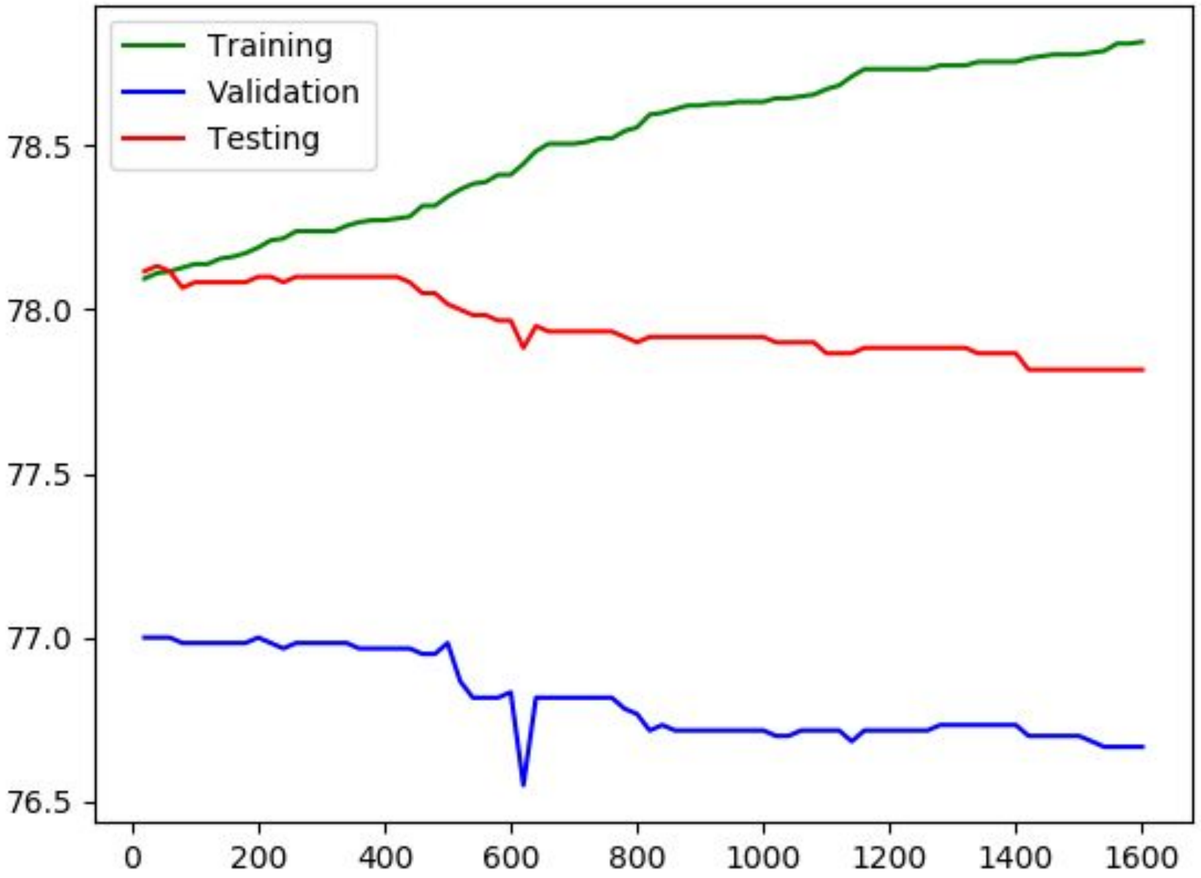
The most common mistake

The `train_test_split` function in scikit-learn can mislead practitioners into treating validation as optional or unimportant.

As a consequence, the **“test” set** is frequently **treated as a validation set**, which removes the possibility of a proper final test evaluation.



Example of test as validation



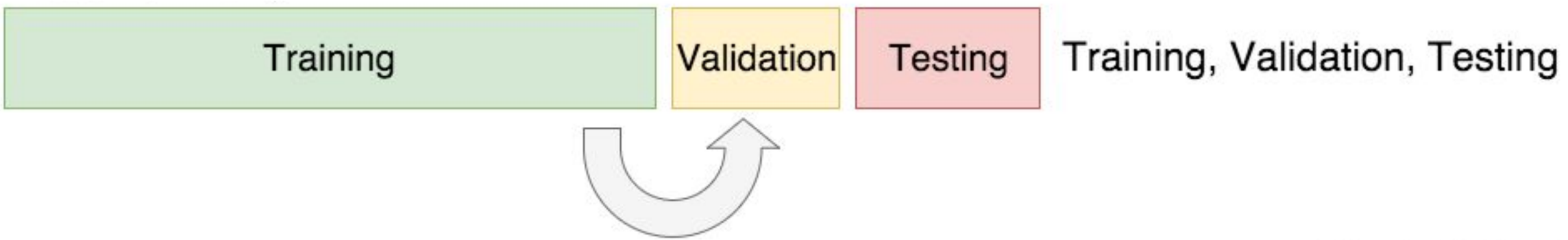
Even in popular sources test set is evaluated on each iteration which makes **test set become one more validation set**

https://www.geeksforgeeks.org/machine-learning/difference-between-model-parameters-vs-hyperparameters/

Fixed dataset split method

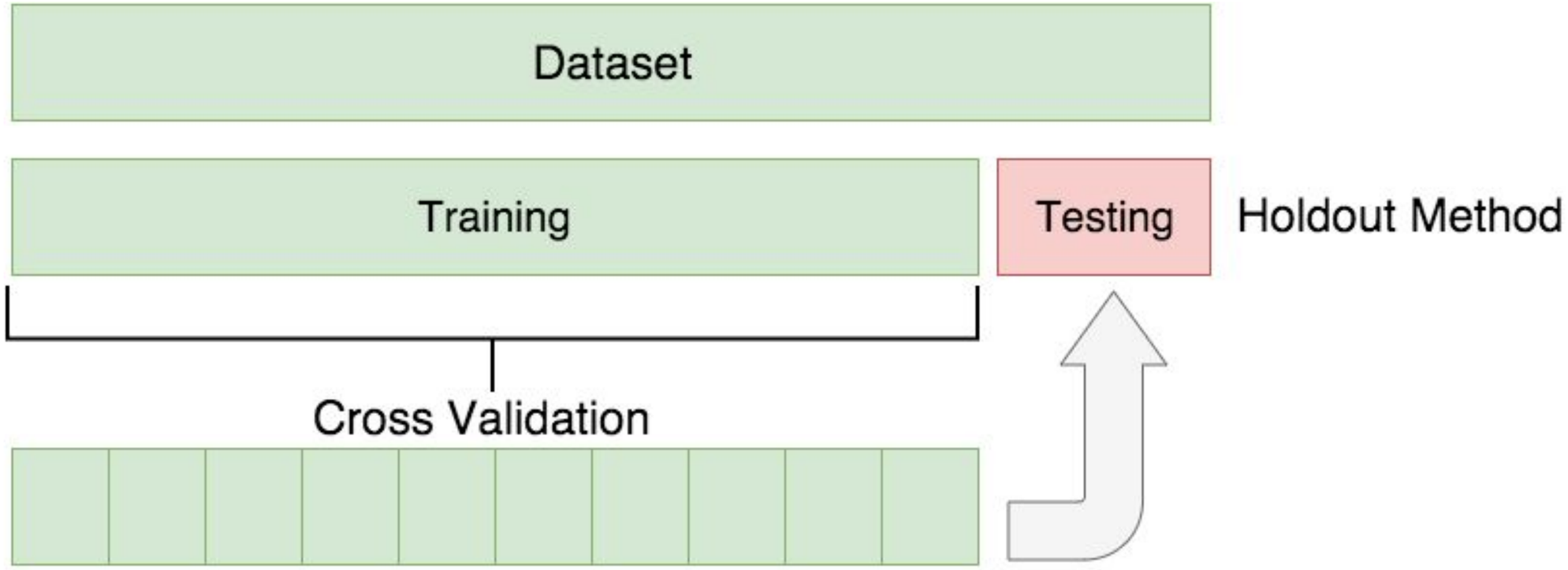
Here we split existing data once at the start and use these sets for all further experiments.

Fixed dataset splits are the predominant approach in practice today, since modern datasets are generally large enough to reserve separate validation and test subsets.



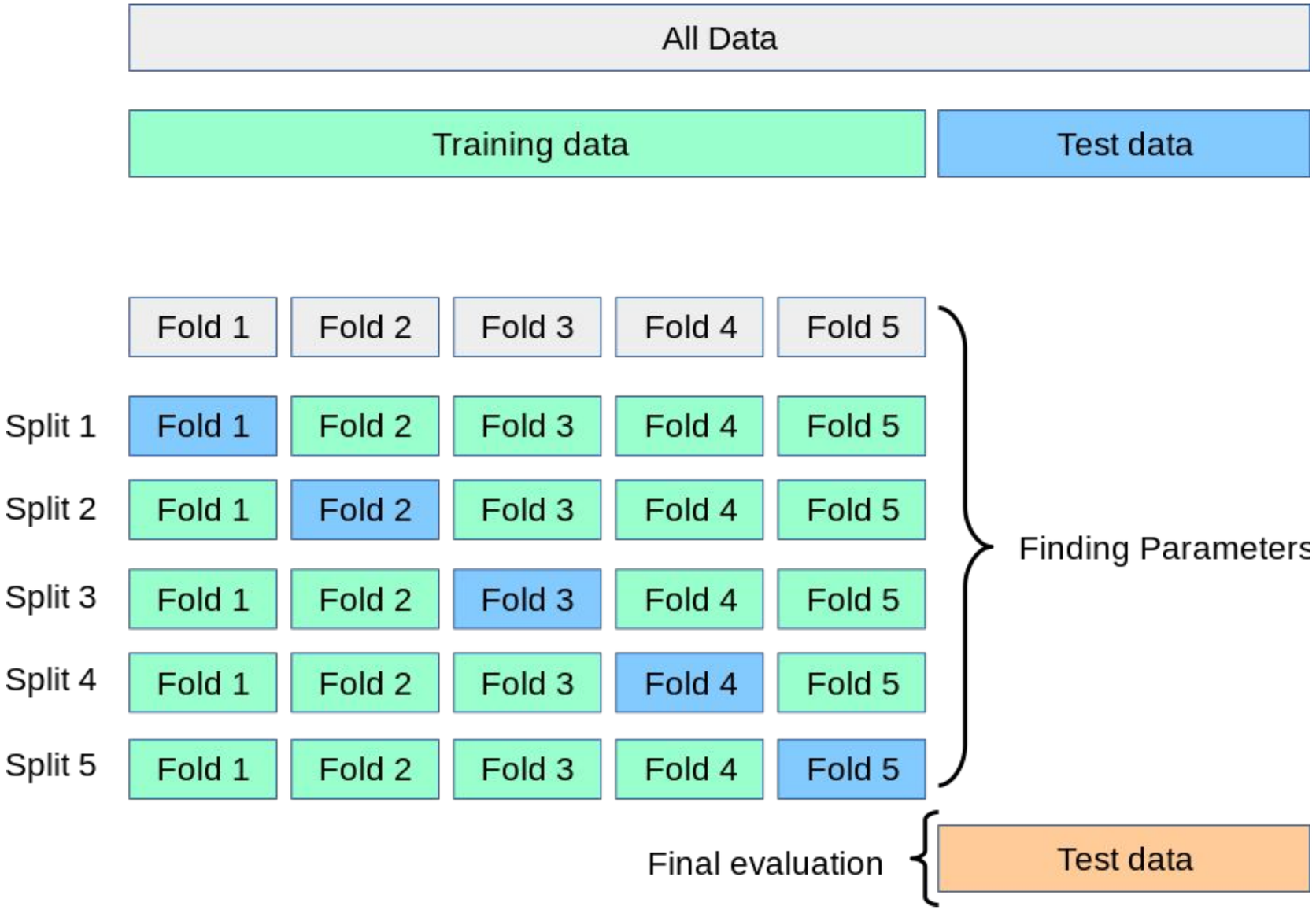
But for small datasets one want to be more efficient in use of the available data

Cross-validation method



In real life is used only on **small datasets** (<10^4 samples)

K-fold cross-validation algorithm



Stages organization



- fixed splits
  - works when fixed val and test samples are big enough to be representative
  - so today used in the vast majority of cases (~99%)
- cross-validation
  - efficient on small datasets because it makes validation set equal to train
  - used to be popular at age of small datasets
  - thought in most machine learning courses
  - edge case is Leave One Out (LOO) where number of splits equals to number of objects
- custom methods
  - whatever you compose for your case
  - can be pretty complex in some cases
    - e.g. uplift contain 5 validation sets

Size of splits

Subject to ad-hoc solutions in most of the cases. Split size controls tradeoff between training error and variance of performance estimation.

Dataset size	Recommended method	Typical ratio	Rationale
< 1,000 samples	K-fold CV (k=5–10)	80/20	Reduce variance due to small sample size
1k–100k	Train/val/test	60/20/20	Enough test data for stable estimate
100k–10M	Train/val/test	80/10/10 or 90/5/5	Data abundant; prioritize training
>10M	Holdout split	95/5 or 98/2	Test variance negligible

Validation summary



There are *two independent decisions* regarding the validation process:

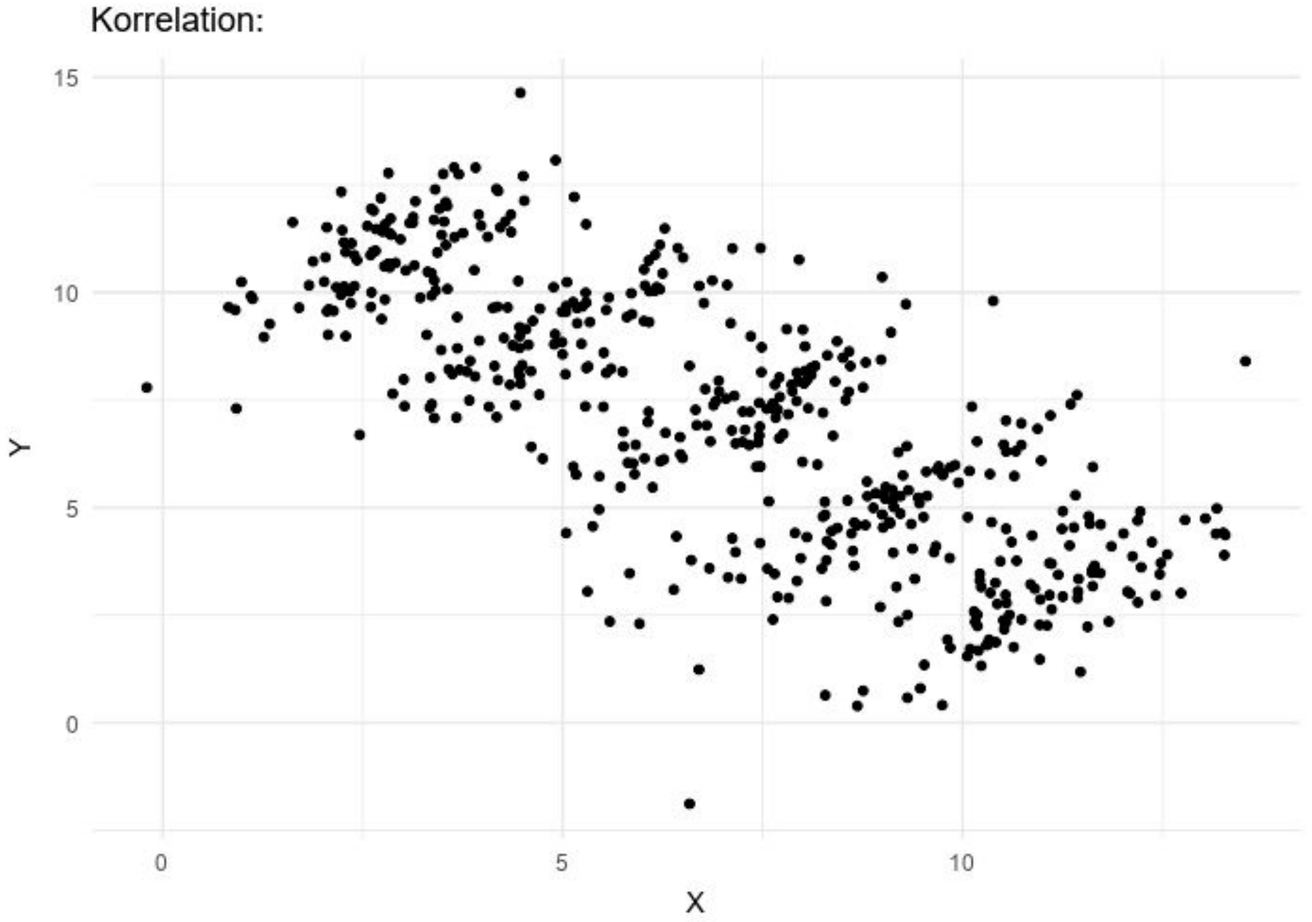
1. Split method
    a. uniform random
    b. stratified
    c. grouped
    d. timeseries
    e. custom
2. Stages organization
    a. fixes splits
    b. cross-validation
    c. custom
3. Split ratio
    a. big for small datasets
    b. small for big datasets

Several validation sets

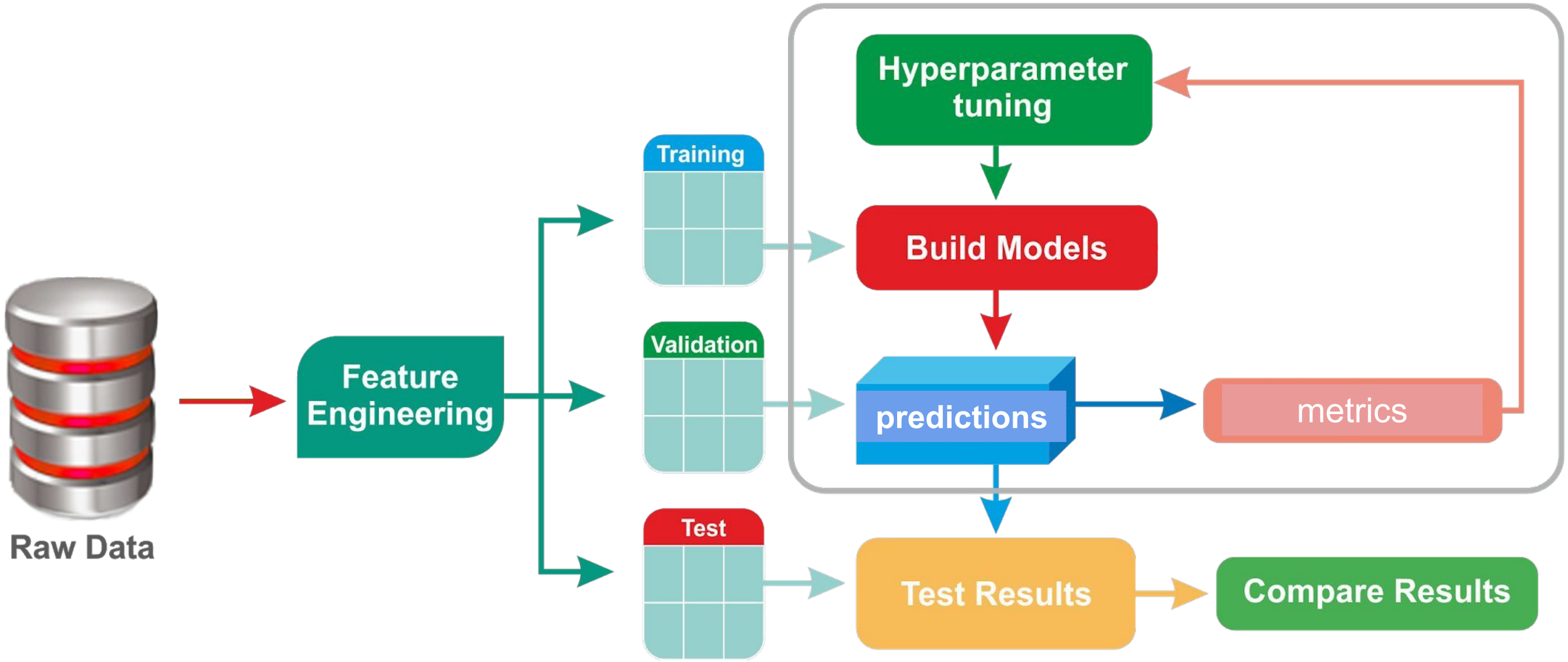
For different auditory

- sex
  - men
  - women
- age
  - older
  - younger

Canonical statistical example is [Simpson's paradox]



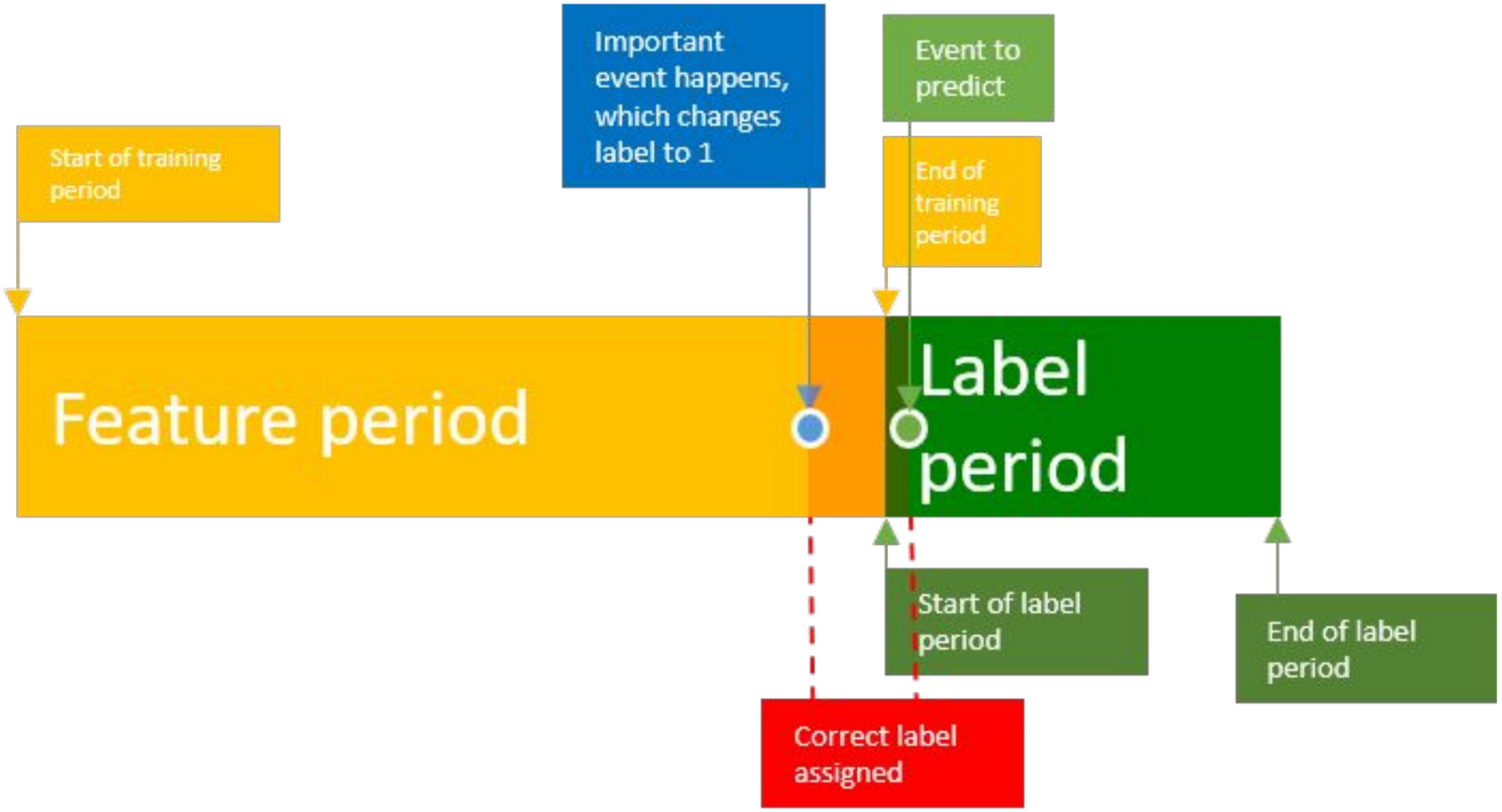
Stages of model training



Although feature engineering better be done after split!

Data drift

Improperly organized validation and evaluation may result in unfair information usage by the model!



Revise

1. Linear models overview
2. Linear Regression under the hood
3. Gauss-Markov theorem
4. Regularization in Linear regression
5. Model validation and evaluation

Thanks for attention!

Questions?



girafe
ai

